

TOPIC :- Licence Plate Detection and Recognition Using Python.

#Code

Expanded dictionary mapping ALL major state codes to state names

state_codes = {

*"AN": "Andaman and Nicobar Islands", "AP": "Andhra Pradesh", "AR":
 "Arunachal Pradesh",*

"AS": "Assam", "BR": "Bihar", "CH": "Chandigarh", "CG": "Chhattisgarh",

*"DD": "Dadra and Nagar Haveli and Daman and Diu", "DL": "Delhi", "GA":
 "Goa",*

*"GJ": "Gujarat", "HR": "Haryana", "HP": "Himachal Pradesh", "JK":
 "Jammu and Kashmir",*

"JH": "Jharkhand", "KA": "Karnataka", "KL": "Kerala", "LA": "Ladakh",

*"LD": "Lakshadweep", "MP": "Madhya Pradesh", "MH": "Maharashtra",
 "MN": "Manipur",*

"ML": "Meghalaya", "MZ": "Mizoram", "NL": "Nagaland", "OD": "Odisha",

"PB": "Punjab", "PY": "Puducherry", "RJ": "Rajasthan", "SK": "Sikkim",

*"TN": "Tamil Nadu", "TR": "Tripura", "TS": "Telangana", "UK":
 "Uttarakhand",*

"UP": "Uttar Pradesh", "WB": "West Bengal"

}

=====

IMAGE PATH

=====

image_path = r"images/car2.png"

```
# =====  
  
# LOAD IMAGE  
  
# =====  
  
img = cv2.imread(image_path)  
  
  
if img is None:  
    print("\nERROR: Unable to read image. Please check the path.")  
    exit()  
  
  
print("Image loaded successfully.\n")  
  
  
  
# Resize for uniform processing  
img = imutils.resize(img, width=600)  
original_image = img.copy()  
h_img, w_img, _ = img.shape  
  
  
  
  
# =====  
  
# PRE-PROCESSING  
  
# =====  
  
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)  
bfilter = cv2.bilateralFilter(gray, 11, 17, 17)  
edged = cv2.Canny(bfilter, 30, 200)  
  
  
  
  
# =====  
  
# FIND CONTOURS  
  
# =====
```

```

keypoints = cv2.findContours(edged.copy(), cv2.RETR_TREE,
cv2.CHAIN_APPROX_SIMPLE)

contours = imutils.grab_contours(keypoints)

contours = sorted(contours, key=cv2.contourArea, reverse=True)[:15]


location = None

cropped_image = None


# Try standard contour bounding detection
for contour in contours:

    peri = cv2.arcLength(contour, True)
    approx = cv2.approxPolyDP(contour, 0.02 * peri, True)

    if 4 <= len(approx) <= 6:
        x, y, w, h = cv2.boundingRect(contour)
        aspect_ratio = float(w) / h
        if 2.0 <= aspect_ratio <= 6.0:
            location = approx

            # Crop with an 8-pixel padding margin so characters aren't clipped on
edges
            cropped_image = gray[max(0, y-8):min(h_img, y+h+8), max(0,
x-8):min(w_img, x+w+8)]

            print("Target plate area localized via contour tracking.")
            break


# =====
# REGION OF INTEREST (ROI) FALLBACK SCANNING
# =====

```

```

if cropped_image is None or cropped_image.size == 0:
    print("Contour geometric search failed. Deploying predictive frame fallback...")
    roi_ymin = int(h_img * 0.35)
    roi_ymax = int(h_img * 0.90)
    roi_xmin = int(w_img * 0.10)
    roi_xmax = int(w_img * 0.90)

    cropped_image = gray[roi_ymin:roi_ymax, roi_xmin:roi_xmax]
    location = np.array([[[roi_xmin, roi_ymin]], [[roi_xmax, roi_ymin]],
[[roi_xmax, roi_ymax]], [[roi_xmin, roi_ymax]]])

if cropped_image is None or cropped_image.size == 0:
    print("Error: Could not extract an evaluation window.")
    exit()

# =====
# HIGH-DEFINITION IMAGE ENHANCEMENT PIPELINE
# =====

# 1. Upscale image significantly so the character edges become clean and
distinct

cropped_image = cv2.resize(cropped_image, None, fx=3, fy=3,
interpolation=cv2.INTER_CUBIC)

# 2. Apply a Sharpening Filter to make text edges pop out distinctly
sharpen_kernel = np.array([[-1,-1,-1], [-1,9,-1], [-1,-1,-1]])
crop_sharp = cv2.filter2D(cropped_image, -1, sharpen_kernel)

# 3. Apply Adaptive Thresholding

```

```
crop_adaptive = cv2.adaptiveThreshold(crop_sharp, 255,  
cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 2)
```

```
# Save intermediate files for direct diagnostic visibility
```

```
cv2.imwrite("debug_crop_sharp.jpg", crop_sharp)
```

```
cv2.imwrite("debug_crop_adaptive.jpg", crop_adaptive)
```

```
# =====
```

```
# OCR MULTI-PASS READING
```

```
# =====
```

```
print("Reading number plate data...")
```

```
reader = easyocr.Reader(['en'], gpu=False)
```

```
result = []
```

```
# Pass 1: Try reading the sharpened grayscale crop (Best for OCR text fonts)
```

```
result = reader.readtext(crop_sharp)
```

```
# Pass 2: Fallback to adaptive thresholding if pass 1 drops characters
```

```
if len(result) == 0:
```

```
    result = reader.readtext(crop_adaptive)
```

```
# Pass 3: Fallback to original raw crop scale
```

```
if len(result) == 0:
```

```
    result = reader.readtext(cropped_image)
```

```
# =====
```

```
# CHARACTER EXTRACTION & CLEANUP
```

```

# =====

plate_text = ""

if len(result) > 0:
    # Compile fragmented character matches safely
    plate_text = "".join([detection[1] for detection in result])
    # Remove spacing anomalies and illegal non-alphanumeric punctuation
    marks
    plate_text = "".join(c for c in plate_text if c.isalnum()).upper()
    print("\n--> Detected Number Plate:", plate_text)
else:
    print("\n[OCR Failure]: No text strings resolved inside evaluation crops.")
    exit()

# =====

# GEOGRAPHIC STATE MAPPER

# =====

state_code = plate_text[:2]
state_name = state_codes.get(state_code, "Unknown State")
print("--> Associated Territory/State:", state_name)

# =====

# WRITE METADATA OUTPUTS

# =====

display_text = f"{plate_text} | {state_name}"
cv2.drawContours(img, [location], -1, (0, 255, 0), 3)

```

```
cv2.putText(  
    img,  
    display_text,  
    (15, 35), # Fixed coordinate placement ensures text never clips outside  
rendering window limits  
    cv2.FONT_HERSHEY_SIMPLEX,  
    0.8,  
    (0, 255, 0),  
    2,  
    cv2.LINE_AA  
)
```

```
# Export parsed sequence values to CSV spreadsheet  
with open("detected_plates.csv", "a", newline="") as file:
```

```
    writer = csv.writer(file)  
  
    writer.writerow([plate_text, state_name, datetime.now().strftime("%Y-%m-%d  
%H:%M:%S")])
```

```
# Save master visualization output image
```

```
cv2.imwrite("result.jpg", img)
```

```
print("\nExecution finished successfully. Records saved to 'detected_plates.csv'  
and 'result.jpg'.")
```

#OUTPUT:



UP1LDX123L | Uttar Pradesh



MH12SR1234 | Maharashtra



DL12CT3456 | Delhi

